What the 010 Trigenic Model Is Measured Against

Additive nulls, what each model actually sees, and whether the comparison is fair

Michael Volk

September 9, 2026

Abstract

A model that claims to learn genetic interaction has to beat a null that cannot represent interaction. Fitting that null on experiment 010 puts a per-gene additive ridge at 0.400 test Pearson against the cell graph transformer's 0.438 to 0.455, and puts a model that sees only which recurring gene pair a triple contains, ignoring the third gene entirely, at 0.390. Reading the model code establishes what the comparison is between: the transformer's encoder never sees the strain, the nine interaction graphs enter training through a penalty on layer-1 attention rather than through the function that maps a checkpoint to a prediction, and the strain-dependent computation is a set function over the three perturbed gene identities, which is the same input and the same hypothesis-class shape the nulls have. That penalty was not incidental: it carried 99.99 percent of the training loss, the regularized heads recover their graphs at 0.83 to 1.00 recall, and a matched pair of runs with the penalty switched off fails to train at all. Rebuilding the forward pass reproduces the recorded metrics to the fourth decimal and yields per-record predictions. Those show the transformer is not a relabeled additive model, correlating with it at 0.73 to 0.76 rather than the 0.999 reported for MULTI-evolve, and that its gain over the null is +0.04 to +0.055 Pearson with bootstrap intervals excluding zero. Residualizing each prediction on the additive model isolates the non-additive content, which is 42 to 47 percent of prediction variance, agrees between two training runs at 0.53 to 0.58, and tracks the part of the label the additive model misses at 0.23 to 0.25.

The result that changes what the metric means is the split. Every record in this build carries one of 420 Kuzmin query doubles, and the published split is random over records, so the same query double appears in train and test. Refitting on a query-pair-disjoint split collapses the additive null from 0.400 to 0.127 ± 0.033 across five folds and inverts the ladder, with the nonlinear baseline falling below the additive one. Separately, the predictions over the unmeasured design space that produced the panel selection are invalid: the genome gene set shrank from 6,607 to 6,579 between inference runs, and the 28 missing mitochondrial open reading frames sort first, so every gene index was shifted by 28 and each triple was scored as a different triple.

Section status: ✓ ready ■ author-reviewed, keep stable ✗ not done, agents may edit freely

Provenance: ▲ not in the library mirror; verify at the linked source ● from a review's summary table, not the primary paper

Contents

1 The question ✗

Tran et al. introduced MULTI-evolve in *Science* in 2026 (`10.1126/science.aea1820`), a protein engineering framework whose fully connected networks were said to "learn the epistatic landscape" from single and double mutants and to identify synergistic combinations at higher order. Visani, Verma and DeWitt replied in a preprint, *Additive baselines furnish no evidence for epistasis learning by MULTI-evolve* (bioRxiv `2026.04.23.719915`), which does one thing: it fits the null that the original paper never fit.

An **additive model** over mutations predicts a phenotype as a sum of per-mutation terms. By construction it assigns exactly zero to every interaction. It is therefore the natural null for any claim that a model has learned interaction, in the same way that an intercept-only model is the null for a claim that a model has learned anything.

Their result is that the network's predictions across the combinatorial variant space correlate with a ridge-regularized additive model's at $r > 0.999$ for all three engineering campaigns, that the variants selected for construction are ranked nearly identically by both, and that on held-out test data the network does not significantly outperform either the additive model or a zero-hidden-layer version of its own architecture. The engineering result is real; the attribution to epistasis learning is not supported.

The analogous question for experiment 010 is whether the equivariant cell graph transformer's held-out performance on the trigenic interaction score survives the same test. Traditional machine-learning baselines exist in this repository for the mixed digenic and trigenic set under `experiments/002-dmi-tmi` and for fitness under `experiments/smf-dmf-tmf-001`. None had been fit on the 010 build, which is triples only.

One difference in the setup, and it cuts against the null. In the protein setting the quantity predicted is the multimutant phenotype itself, which per-mutation effects explain directly. In 010 the quantity predicted is already a residual. The trigenic interaction score subtracts the expectation built from single and double mutant fitness before the model ever sees it, so the additive part of fitness has been removed by construction. An additive-in-single-gene-effects model is therefore a harder null here than it is there, and its performance is more surprising, not less.

That argument has one loophole, and quantifying it turns out to matter more than anything else in this document. It is taken up in Section 6 as baseline B4.

2 What is being predicted ✗

2.1 The label ✗

Write f_i for the fitness of the single mutant Δi, f_{ij} for the double, f_{ijk} for the triple, each relative to wild type. The **digenic interaction score** is the departure of the double from the multiplicative expectation,

$$\varepsilon_{ij} \;=\; f_{ij} - f_i f_j, \tag{1}$$

and the **trigenic interaction score** is the departure of the triple from the expectation built out of the singles *and* the doubles,

$$\tau_{ijk} \;=\; f_{ijk} \;-\; f_i f_j f_k \;-\; f_i\,\varepsilon_{jk} \;-\; f_j\,\varepsilon_{ik} \;-\; f_k\,\varepsilon_{ij}. \tag{2}$$

▲ [external: Kuzmin et al. 2018, `10.1126/science.aao1729`; the published definition of the released adjusted score column, not re-derived here]

Two properties of Eq. 2 matter for everything that follows. It is a *residual*: the additive-on-the-log-scale, multiplicative-on-the-linear-scale part of fitness is subtracted before the number is reported. And it is a *third-order* residual: the pairwise terms are subtracted too, so τ is by construction the part of the triple mutant phenotype that neither single nor double mutant measurements predict.

The 010 loader consumes the column `Adjusted genetic interaction score (epsilon or tau)` from the Kuzmin releases, which for a trigenic record is τ, and stores it under the label key `gene_interaction`.

2.2 The build ✗

The build is triples only, so there are no singles or doubles in it. The MULTI-evolve experiment of training on low order and extrapolating to high order is therefore not available here, and the question is the narrower one: given a training set of triples, how much of the held-out τ is recoverable without representing interaction.

Records	376,732
Perturbation count	3 for every record
Distinct perturbed genes	4,352
Sources	`TmiKuzmin2018Dataset`, `TmiKuzmin2020Dataset`
Label	`gene_interaction` (τ)
Split, seed 42	train 301,386 / val 37,673 / test 37,673
Train label SD	0.063186
Val label SD	0.062914
Test label SD	0.064187

Table 1: The 010 build. Every baseline and the transformer use these records, this split and this label. Read from `001-small-build/processed/label_df.parquet`, `is_any_perturbed_gene_index.json` and `data_module_cache/index_seed_42.json`.

2.3 The screen design, which is the confounder ✗

Kuzmin's trigenic screen crosses a **query** double mutant against an array of single mutants, so a triple is a query pair plus one array gene, and the same query pair recurs across hundreds or thousands of triples. Counting unordered gene pairs in the training split finds 593,978 distinct pairs, of which only 420 occur five or more times, the most frequent occurring 3,308 times. That concentration is the query-pair structure.

The 010 split is random over records, so a query pair that appears in training also appears in validation and test. Any model that can represent a per-pair offset can absorb whatever is common to a query batch, and none of that is trigenic biology. Baseline B4 in Section 4 is built to measure exactly this quantity.

3 What each model actually sees ✗

The comparison is only meaningful if the models are given the same information. Reading the 010 code settles that, and the answer is stronger than expected: every model in this document, the transformer included, receives exactly one thing per record, the set of three perturbed genes.

3.1 The transformer's per-sample input ✗

The 010 dataset is built with the `Perturbation` graph processor. That processor writes the perturbed gene names, their indices, a boolean mask and the label into each sample, and nothing else (`graph_processor.py`, lines 1984 to 2058). There is no per-gene feature matrix: with an empty `node_embeddings` list in the config, the gene feature tensor of the cell graph has shape 6607×0 (`cell_data.py`, line 53). Gene identity is carried by a free lookup table of 6,607 rows and 180 columns (`equivariant_cell_graph_transformer.py`, line 1989), initialized randomly and fit by gradient descent.

No sequence, expression, ontology or other precomputed representation enters. The model is given gene identities, the same as a one-hot design matrix, and learns its own 180-dimensional vector for each.

3.2 The encoder does not depend on the strain ✗

The forward pass builds its input as `gene_embedding(torch.arange(N))`, prepends a CLS token, and unsqueezes to shape $[1, N{+}1, 180]$. That leading dimension is 1, not the batch size. The eight transformer layers then run on this single tensor, which is assembled entirely from parameters and contains nothing from `batch`. Writing E for the embedding table and T for the eight-layer encoder,

$$Z \;=\; T(E) \in \mathbb{R}^{N \times 180}, \qquad h_{\mathrm{CLS}} \in \mathbb{R}^{180}, \tag{3}$$

and Z and h_{CLS} are the same for every record in the dataset. The repository states this itself, in the docstring of `PerturbationGraphPropagation`: "The encoder runs at batch 1 on the WILDTYPE graph, so `H_genes` and `h_CLS` are identical for every strain; the only strain-dependent step is `EquivariantPerturbationTransform`."

So the eight encoder layers, which hold 3,129,120 of the model's 4,774,861 parameters, perform no per-sample computation. They map one free parameter block to another. Whatever role they play, it is not to process the strain.

3.3 Where the nine graphs act, and where they do not ✗

The model has three places adjacency could enter the computation that produces a number. In the 010 configuration all three are off. The attention-mask configuration is not passed, so the head mask is `None` and every layer receives no mask. The perturbation-propagation configuration is not passed either, so that block is skipped. The attention itself is ordinary softmax self-attention with no adjacency bias term. Given a trained checkpoint, then, no adjacency tensor is an input to any operation that yields $\hat{y}$.

That is a much narrower statement than it first appears, and an earlier version of this document overstated it. The nine adjacency matrices are consumed by a Kullback-Leibler penalty between each row-normalized adjacency $\tilde{A}$ and one attention head of layer 1,

$$\mathcal{L}_{\text{graph}} = \sum_{k=1}^{9} c_k \sum_{i=1}^{N}\sum_{j=1}^{N} \tilde{A}_{ij}^{(k)} \log \frac{\tilde{A}_{ij}^{(k)}}{\alpha_{ij}^{(k)}}, \qquad \alpha^{(k)} = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d}}\right)_{h_k}, \tag{4}$$

and that penalty is computed *inside* the same forward call as the prediction, from the same pre-dropout attention tensor. Nothing detaches it. Its gradient therefore reaches the layer-1 query and key projections, through them every layer-0 parameter, and through those the gene embedding table. The trained $Z = T(E)$ of Eq. 3 is shaped by the nine graphs even though Z's value at inference is read off fixed weights.

So the correct statement is not that the graphs are absent. It is that they act on the model's *parameters* during training and not on the *function* from checkpoint to prediction. An ablation that zeroes the coefficient on a trained checkpoint tests only the second, and Section 6.2 reports it as such.

The penalty was not weak, it dominated. Reconstructing the coefficient matters because the earlier version of this document reported it as negligible on two counts, and both were wrong for these runs. The per-graph weight c_k in Eq. 4 is a product of three factors: the per-head $\lambda_k = 10^{-3}$, a global scale of 1.0 read from `regression_task.loss.graph_regularization.lambda`, and an edge normalization intended to divide by mean degree. That last factor is computed as `len(A.nonzero()[0])` summed over the nine matrices. On a dense `torch` tensor `A.nonzero()` has shape $[\text{nnz}, 2]$, so `[0]` is a single index pair of length two, and the sum over nine graphs is 18 rather than the 2.5 million edges intended. Dividing by 18/6607 multiplies by 367.06, giving

$$c_k = 10^{-3} \times 1.0 \times \frac{6607}{18} = 0.367, \tag{5}$$

applied to a Kullback-Leibler divergence summed over all 6,607 rows. The runs' own logs confirm the consequence. Against a point loss of 0.81, the graph term is 5,740, and the trainer's own diagnostic `norm_weighted_graph_reg`, which is the graph term divided by the total loss, reads 0.99986 for all three checkpoints. The graph penalty was 99.99 percent of what the optimizer minimized.

Two claims withdrawn. The earlier version stated that the configuration named `physical` and `regulatory` while the cell graph emitted `physical_interaction` and `regulatory_interaction`, leaving two heads unregularized, and that the weight was applied twice from one configuration key for an effective 10^{-6}. Both are false for the 010 runs. The suffixed edge names were introduced on 2026-06-27, six months after these runs, and at the December 2025 commit the cell graph emitted the bare names the configuration used; the runs log `val_edge_recovery/physical_L1_H0` and `regulatory_L1_H1`, keys the trainer emits only for a graph it matched, so all nine heads were regularized. The squared-lambda defect is real but belongs to the 019 configuration, where both keys interpolate the same 10^{-3}; the 010 training script reads the global scale from a different key whose value is 1.0. The July 2026 commits did fix both defects, but neither defect was operating here.

3.4 The transformer is a set function on three genes ✗

What remains after Eq. 3 is the strain-dependent part. For a record with perturbed set S, $|S| = 3$, the perturbation transform performs cross-attention in which all N genes query the $|S|$ rows $\{Z_p : p \in S\}$, followed by a residual, a layer norm and a feedforward block, giving $H_i^{\text{pert}} = g(Z_i + c_S)$ where c_S is the attended context. The head then pools over the perturbed genes and reads out,

$$z_S = \frac{1}{|S|}\sum_{i \in S} H_i^{\text{pert}}, \qquad \hat{y} = \text{MLP}([\, h_{\text{CLS}} \,\|\, z_S \,]), \tag{6}$$

with the MLP being $\text{Linear}(360, 180) \to \text{ReLU} \to \text{Linear}(180, 1)$.

Since h_{CLS} is constant, Eq. 6 is a function of z_S alone, and z_S is a permutation-invariant pooling of quantities derived from $\{Z_p : p \in S\}$. The 010 transformer is therefore a set function on the three perturbed gene identities, built on a

free per-gene vector. That is the same functional shape as the additive nulls, which are set functions on the same three identities built on a free per-gene scalar. The models differ in how expressive the set function is, not in what they are allowed to see.

This is what makes the comparison in Section 6 fair. It does not settle where the transformer's advantage over the nonlinear baseline comes from. An earlier version of the working note assigned that gap to the nine interaction graphs, and a later one assigned it to capacity on the grounds that the graphs are absent from the prediction. Both overreached. The graphs shaped the weights this set function is built from, and Section 6.3 measures how heavily.

4 The models ✗

4.1 Where each baseline comes from ✗

Two of the six baselines are taken from the Visani preprint. The rest were built here for a confounder that the protein setting does not have. Naming which is which matters, because a baseline borrowed from a published critique carries that critique's authority and one invented here does not.

Model	Origin	Corresponds to
B0	standard	intercept-only null
B1	Visani et al. 2026	their ridge-regularized additive model
B2	this document	B1 extended to observed pairs
B3	this document	unregularized counterpart of B4
B4	this document	the Kuzmin screen-structure confounder
B5	Visani et al. 2026	their zero-hidden-layer control, run in reverse

Table 2: Provenance of the six baselines.

The two external rows come from Visani, Verma and DeWitt, *Additive baselines furnish no evidence for epistasis learning by MULTI-evolve*, bioRxiv `2026.04.23.719915` (`https://doi.org/10.1101/2026.04.23.719915`), which is itself a reply to Tran et al., *Science* 2026, `10.1126/science.aea1820` (`https://doi.org/10.1126/science.aea1820`). B1 is their additive null transplanted onto gene deletions. B5 inverts their zero-hidden-layer control: where they removed nonlinearity from the published architecture, B5 adds the smallest amount of nonlinearity to the additive feature space, so the two bracket the same gap from opposite sides.

B3 and B4 have no counterpart in the preprint because the confounder they measure is specific to a synthetic genetic array. A protein engineering campaign has no analog of a query double mutant recurring across thousands of measured variants. B4 turns out to carry the result that matters most here, so its being ours rather than theirs is worth stating plainly.

4.2 Notation and the interaction operator ✗

Let G be the 4,352 genes appearing in the build and let record n have perturbed set $S_n \subset G$ with $|S_n| = 3$. Encode it as $x_n \in \{0,1\}^{|G|}$ with $x_{n,g} = 1$ exactly when $g \in S_n$, so that $\|x_n\|_1 = 3$ for every record. Write y_n for the trigenic interaction score of Eq. 2.

A model is a set function $f : 2^G \to \mathbb{R}$. Its capacity to represent interaction is measured by finite differences. For distinct genes g, h, k and any S disjoint from them, define

$$\Delta_g f(S) = f(S \cup \{g\}) - f(S), \tag{7}$$

$$\Delta_{gh} f(S) = f(S \cup \{g,h\}) - f(S \cup \{g\}) - f(S \cup \{h\}) + f(S), \tag{8}$$

$$\Delta_{ghk} f(S) = \sum_{U \subseteq \{g,h,k\}} (-1)^{3-|U|} f(S \cup U). \tag{9}$$

$\Delta_{gh} f \equiv 0$ says the model cannot express any pairwise interaction; $\Delta_{ghk} f \equiv 0$ says it cannot express any three-way interaction. These are the properties that make a model a null, and they are structural, holding for every parameter setting rather than for a fitted one.

4.3 B0, the train mean ✗

$\hat{y} = \bar{y}_{\text{train}}$. Every finite difference vanishes. Its mean squared error is the label variance, and it fixes the zero point against which all reductions are read.

4.4 B1, the additive per-gene ridge ✗

$$\hat{y}(S) \;=\; \beta_0 + \sum_{g \in S} \beta_g, \qquad \hat{\beta} \;=\; \arg\min_{\beta} \sum_{n \in \text{train}} \left(y_n - \beta_0 - x_n^\top \beta\right)^2 + \alpha \|\beta\|_2^2, \tag{10}$$

with β_0 unpenalized and set to $\bar{y}_{\text{train}}$. Substituting Eq. 10 into Eq. 8 gives $\Delta_{gh}\hat{y} = 0$ and hence $\Delta_{ghk}\hat{y} = 0$ identically. B1 is the direct analog of the additive null in the DeWitt preprint.

The design is well conditioned despite having 4,352 coefficients: with three ones per row and 301,386 training rows there are 904,158 gene observations, a median of 9 per gene in the training split with a maximum of 3,430. The ridge penalty is chosen on validation over $\alpha \in \{10^{-2}, 10^{-1}, 1, 3, 10, 30, 100, 300, 1000\}$ and the selected value is then reported on test.

4.5 B2, additive plus recurring gene pairs ✗

Let P be the set of unordered gene pairs occurring at least five times in the training split, $|P| = 420$. Adding one coefficient per such pair,

$$\hat{y}(S) \;=\; \beta_0 + \sum_{g \in S} \beta_g + \sum_{\{g,h\} \subseteq S,\ \{g,h\} \in P} \gamma_{gh}, \tag{11}$$

gives $\Delta_{gh}\hat{y} = \gamma_{gh}$ for a pair in P and zero otherwise. B2 therefore has second-order capacity on observed pairs. It still has *no* third-order capacity: $\Delta_{ghk}\hat{y} = 0$ identically. Since the label is itself a third-order residual, B2 remains a null with respect to the quantity being predicted.

4.6 B3 and B4, the screen-structure baselines ✗

B4 drops the per-gene terms from Eq. 11 and keeps only the pair terms,

$$\hat{y}(S) \;=\; \beta_0 + \sum_{\{g,h\} \subseteq S,\ \{g,h\} \in P} \gamma_{gh}. \tag{12}$$

Because the recurring pairs are the Kuzmin query doubles, B4 is close to a per-query-batch offset. For a triple made of a query pair and an array gene, the array gene contributes nothing at all to Eq. 12. Whatever B4 achieves is therefore an upper bound on how much of the metric is explained by knowing which screen a record came from.

B3 is the unregularized empirical counterpart: predict the mean training label over triples sharing each recurring pair, backing off to the mean over triples containing each gene, and then to the global mean.

4.7 B5, nonlinearity on the same features ✗

Give each gene a free vector $e_g \in \mathbb{R}^{128}$, sum over the perturbed set and read out with a two-hidden-layer network ϕ,

$$\hat{y}(S) \;=\; \phi\left(\sum_{g \in S} e_g\right), \qquad \phi : \mathbb{R}^{128} \to \mathbb{R}. \tag{13}$$

This is permutation invariant and uses precisely B1's feature space, the identity of the three genes and nothing more. Because ϕ is nonlinear, $\Delta_{gh}\hat{y}$ and $\Delta_{ghk}\hat{y}$ are not identically zero, so B5 is not a null. It is the smallest departure from B1 that has interaction capacity, and it isolates what capacity alone buys.

Comparing Eq. 13 with Eq. 6 shows they are the same construction. Both give every gene a free vector, pool over the three perturbed genes in a permutation-invariant way, and pass the pooled vector through a small network. The transformer's pooling is richer, since the context c_S is produced by attention over the perturbed set rather than by a plain sum, and its per-gene vectors are $Z = T(E)$ rather than a raw table. Trained with three seeds, B5 reports a spread rather than a single number.

4.8 Summary of capacity ✗

Model	Per-gene representation	Set aggregation	$\Delta_{gh} \not\equiv 0$	$\Delta_{ghk} \not\equiv 0$
B0	none	none	no	no
B4	none	recurring pair indicators	yes	no
B3	scalar mean	pair mean, backing off	yes	no
B1	free scalar	sum	no	no
B2	free scalar	sum plus pair terms	yes	no
B5	free 128-vector	sum, then MLP	yes	yes
CGT	free 180-vector via $T(E)$	attention over S, mean, MLP	yes	yes

Table 3: What each model can represent. The last two columns are structural properties of the hypothesis class, not fitted results. Only B5 and the cell graph transformer can express a three-way interaction, which is what the label is.

5 Is the comparison fair ✗

Six things have to match before a baseline number can sit in the same table as a transformer number. Table 4 names them and gives the verdict on each; the subsections that follow give the evidence.

#	Verdict	What has to match
1	matches	**Records and split.** All models use the same 376,732 records and the same `index_seed_42.json` partition.
2	matches	**Label.** All predict `gene_interaction`, the published adjusted τ of Eq. 2.
3	matches	**Reported split.** Test is reported for every row, and validation is what every row selected on.
4	matches	**Metric definition and denominator.** Pearson, Spearman and mean squared error over the full 37,673 test records in raw τ units for every row.
5	matches	**Information available.** The identity of the three perturbed genes, and nothing else, for every model (Section 3).
6	**does not**	**Label normalization.** The transformer's standardization constants are fit on all 376,732 records rather than on the training split. The baselines use the training mean only.

Table 4: What has to match for the comparison to be fair. Row 6 is the one mismatch, and it runs in the transformer's favor.

The loss functions also differ between the baselines and the transformer, but that is a difference in what each model was trained to do rather than in how the comparison is scored, so it is not one of the six and is treated separately below.

5.1 Which split the reported numbers come from ✗

Both validation and test numbers are available for the transformer, and the table in Section 6 reports **test**. The distinction matters because the three transformer checkpoints are *best-Pearson* checkpoints, selected on validation across roughly 25 to 64 validation epochs. A validation Pearson chosen that way is the maximum of many draws and is upward-biased as an estimate of capability; its test Pearson is not, because test was not consulted during selection.

The baselines follow the same discipline. B1, B2 and B4 select the ridge penalty on validation and report test. B5 early-stops on validation and reports test. So the test column compares like with like, and the validation column is biased in the same direction for every row.

5.2 What the transformer metrics are ✗

The metrics come from the three checkpoints' re-evaluation runs under `$DATA_ROOT/wandb-experiments`, read from `files/wandb-summary.json` rather than retyped. Two families of key exist and they are not interchangeable.

- `val/gene_interaction/{MSE,RMSE,Pearson}` and the `test/` equivalents are torchmetrics objects updated once per batch and computed at epoch end, so they cover the *full* 37,673 records of each split. They are computed after the inverse transform, in raw τ units.

- `val_sample/Spearman_target_0` and its test counterpart come from a separate path that collects samples for plotting and is capped by `plot_sample_ceiling`. In the evaluation configuration that ceiling is 10^6, above the split size, so no truncation fires and the Spearman is also over the full split. In the *training* configuration the same ceiling is 10,000, so a Spearman quoted from a training run would be over a random 10,000 of 37,673. The numbers here are from the evaluation runs.
- `val/transformed/gene_interaction/*` are the same metrics in standardized units and are not used here.

The baselines compute Pearson, Spearman and mean squared error over the same full splits in raw τ units, so the units and the denominators agree.

5.3 What each model sees ✗

Established in Section 3: the perturbed gene set, and nothing else, for every model in the comparison. No baseline is handicapped by being denied a feature the transformer had, and the transformer is not credited with information it never received.

5.4 Where the comparison is not clean ✗

Label normalization leaks. The transformer trains on standardized labels, and the standardization statistics are fit on *all* 376,732 records rather than on the training split. The transform is constructed from `dataset.label_df`, which iterates every record in the LMDB with no split filter, and it is constructed before the data module splits the data. The logged constants confirm it: mean -0.008024324 and standard deviation 0.063263549 match the all-record statistics to nine digits and do not match the train-only values -0.007688739 and 0.063186255.

The leak is two global scalars, and the train and all-record values differ by about 3×10^{-4} in the mean and 10^{-4} in the standard deviation, so the effect is small. It is nonetheless real, it runs in the transformer's favor, and the baselines do not have it: they use the training mean only. No correction is applied here, and no attempt is made to estimate the size of the advantage.

The loss functions differ, deliberately. The baselines minimize squared error. The transformer minimizes

$$\mathcal{L} \;=\; \lambda_p \mathcal{L}_{\mathrm{MSE}} \;+\; \lambda_d \, S_\epsilon\big(\tfrac{1}{B}\textstyle\sum_b \delta_{\hat{y}_b},\; \tfrac{1}{B}\sum_b \delta_{y_b}\big) \;+\; \lambda_g \mathcal{L}_{\mathrm{graph}}, \tag{14}$$

with $\lambda_p = 1$, $\lambda_d = 0.1$, $\lambda_g = 1$. The middle term is a debiased Sinkhorn divergence with $p = 2$ and blur 0.05 between the empirical measure of the batch's predictions and that of the batch's targets, computed unpaired, which shapes the marginal distribution of the predictions rather than their per-record accuracy. The third term is the graph penalty of Section 3.3.

This is a difference in training objective, not in the evaluation, and it is part of what the transformer is. It is noted because a distributional term can move Pearson and mean squared error in opposite directions, so the two metrics should be read together rather than either alone.

Paired residuals had to be recomputed. The original evaluation runs wrote per-record prediction files to the cluster checkout, and those files are not on this machine. Recomputing them through the stock evaluation script fails on the frozen build, which no longer validates (Section 8). Section 6.2 gives the route around that, and Section 8 gives the migration that removes the obstacle for future work.

5.5 Model size ✗

Component	Parameters	Role
Gene embedding table	1,189,260	the only place gene identity lives
Encoder, 8 layers	3,129,120	strain-independent, see Sec. 3.2
Perturbation transform	391,140	the strain-dependent step
Readout head	65,161	pooling and MLP
CLS token	180	constant
Total	4,774,861	
B1 additive ridge	4,353	one scalar per gene, plus intercept
B5 embedding MLP	623,745	128 per gene, plus a two-layer network

Table 5: Parameter budget. Two thirds of the transformer sits in the encoder, which does no per-sample work, and a quarter in the embedding table. The strain-dependent computation is 456,301 parameters, under a tenth of the model.

6 Held-out results ✗

Model	Interaction capacity	Pearson	Spearman	MSE
B0 train mean	none	0.000	0.000	0.004124
B4 query pair only	pairwise, 420 pairs	0.390	0.362	0.003493
B3 hierarchical mean	pairwise, empirical	0.390	0.362	0.003493
B1 additive per-gene ridge	none	0.400	0.406	0.003460
B2 additive plus pair ridge	pairwise, 420 pairs	0.406	0.412	0.003442
B5 embedding MLP, seed 0	full	0.425	0.417	0.003382
B5 embedding MLP, seed 1	full	0.427	0.421	0.003395
B5 embedding MLP, seed 2	full	0.425	0.419	0.003381
CGT M01 `lzs9pcj3`	full	0.443	0.426	0.003400
CGT M02 `yv4r30bi`	full	0.438	0.424	0.003439
CGT M03 `c7671wgj`	full	0.455	0.426	0.003315

Table 6: Test split, 37,673 records, raw τ units. The interaction-capacity column repeats Table 3: only the last four rows can express a three-way interaction, which is what the label is. Ridge penalties are selected on validation; B5 early-stops on validation; the transformer checkpoints are selected on validation. Test was not consulted by any of them.

6.1 Reading the ladder ✗

The additive null is close, but the transformer does beat it. B1 reaches 0.400 against the transformer's 0.438 to 0.455, so a model that cannot represent any interaction recovers 88 to 91 percent of the correlation. In explained variance the gap is $r^2 = 0.160$ against 0.207 for the best checkpoint, an increment of 0.047 of label variance. In error, the best checkpoint reduces mean squared error 19.6 percent below B0 and B1 already delivers 16.1 percent of that. This is not the DeWitt result, where the network collapsed onto the additive model at $r > 0.999$. The margin here is real. It is also small.

Seed spread is a large fraction of that margin. The three transformer runs span 0.438 to 0.455, a standard deviation of 0.0088. The three B5 seeds span 0.425 to 0.427, a standard deviation of 0.0011. So the transformer's run-to-run spread is eight times the nonlinear baseline's and is roughly a fifth of its own margin over B1. A single checkpoint's number is not the capability.

Capacity closes about half the gap, and the graphs cannot be ruled out. B5 sees exactly B1's features, has no graph and no attention, and reaches 0.426. That closes about half the distance from B1 to the transformer. On mean squared error B5's 0.003382 is better than two of the three checkpoints.

An earlier version of this document attributed the whole remaining B5-to-transformer difference to capacity, on the grounds that the graphs are not in the forward pass. That inference does not hold, and Section 6.3 gives the measurements that break it. The graphs shape the trained weights even though they are absent from the prediction function, and the evidence that they shaped these weights heavily is direct. What can be said from the ladder alone is that richer set aggregation and eight times the parameters are worth at most 0.03 Pearson over B5, and that how much of that 0.03 is the graphs is not separable from these runs.

B4 is the result that changes what the metric means. B4 knows only which of 420 recurring gene pairs a triple contains. It never looks at the third gene. It reaches 0.390 test Pearson, 97 percent of B1's and 86 percent of the best

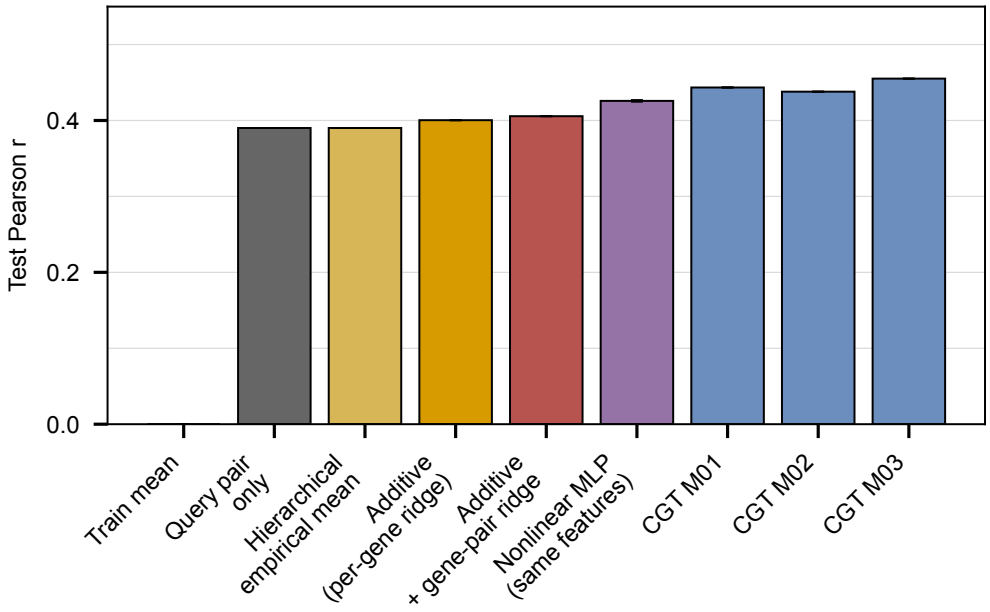

Figure 1: Test Pearson for every model. Error bars on the nonlinear baseline are the standard deviation across three seeds; the transformer bars are three separate training runs shown individually.

checkpoint's. Those 420 pairs are the Kuzmin query doubles, the most frequent appearing 3,308 times, so B4 is close to a per-screen offset.

On a split that is random over records, then, most of what every model in Table 6 is scoring is the identity of the screen a triple came from. That is not trigenic biology and it would not survive a query-pair-disjoint split. It also explains why the additive null does so well on a label that was constructed to have the additive part removed: what B1's per-gene coefficients absorb is largely query-pair structure reaching them through the genes that make up the query pairs.

Spearman compresses the ladder further. The rank correlations run 0.406 for B1, 0.412 for B2 and 0.417 to 0.421 for B5, against 0.424 to 0.426 for the transformer. The gap from the additive null to the transformer is 0.020 in Spearman against 0.045 in Pearson. Ranking is what a design-space selection consumes, so the smaller of the two is the more decision-relevant number.

6.2 Verifying the architecture reading ✗

Section 3 claims the encoder is strain-independent and that no adjacency tensor is an input to the prediction. Both are checkable rather than merely readable, and checking them also produces the per-record predictions that Section 9 lists as missing.

The stock evaluation path cannot run against the frozen build, for the reasons Section 8 sets out. The route around it follows from the architecture. If the encoder really does not see the strain, then scoring a record needs only the perturbed gene indices, and the index space is the sorted base-graph node list, rebuildable from the genome without touching the LMDB. `score_010_checkpoints_directly.py` does that, loads each checkpoint into the model class, and recomputes the metrics.

The check is that the recomputed numbers must reproduce what the original evaluation runs recorded. A wrong gene ordering or a mis-loaded weight would give a plausible but wrong number, so agreement to several digits is the evidence that the reconstruction is faithful.

	Val Pearson		Val MSE	
Checkpoint	recomputed	recorded	recomputed	recorded
M01 `lzs9pcj3`	0.451970	0.451963	0.003244	0.003244
M02 `yv4r30bi`	0.447319	0.447155	0.003258	0.003259
M03 `c7671wgj`	0.461917	0.461881	0.003163	0.003163

Table 7: Scoring the checkpoints through a hand-built forward pass that computes the encoder once and feeds it only perturbed-gene indices reproduces the recorded metrics. The residual difference is at the fourth decimal and is consistent with the original runs using mixed bf16 precision.

Two details had to be right for this to work, and both are worth recording because getting either wrong produced a

plausible but wrong answer. The index space is the sorted genome gene set, 6,607 genes, not the build's own 6,579 gene set; using the latter gave a validation Pearson of −0.001 to 0.002. And the readout pools by *mean*, whereas the current class defaults to `sum`; using the default gave 0.420 to 0.438, close enough to look credible and wrong.

That reproduction is the direct verification of Section 3.2. The encoder was evaluated exactly once, on a strain-independent input, and the recorded per-split metrics came back. If the encoder had needed the strain, this could not have happened.

The other claim needs a sharper statement than it was given. Setting the graph penalty weight from 1.0 to 0.0 on a trained checkpoint changes the maximum absolute test prediction by 9.0×10^{-7}, 1.2×10^{-6} and 1.3×10^{-6}, against a prediction spread of about 0.036. That establishes what it establishes: adjacency is not an input to the prediction function. It says nothing about whether the graphs shaped the weights being scored, and Section 6.3 shows they did.

6.3 What the graphs did during training ✗

Three independent measurements, none of which the inference-time ablation can see.

The penalty was almost the whole objective. The trainer logs the graph term, the point term and their normalized shares. Against a validation point loss of 0.81 the graph term is 5,740, and the share `norm_weighted_graph_reg` is 0.99986 for all three checkpoints. Eq. 5 accounts for the size: the intended division by mean degree is a multiplication by 367, and the resulting per-graph coefficient is 0.367 on a divergence summed over 6,607 rows.

The regularized heads recover their graphs. The trainer scores each regularized head by asking, for every gene i with degree d_i, what fraction of i's true neighbors fall in that head's top d_i attention targets.

Graph	Head	M01	M02	M03
physical	0	0.940	0.940	0.938
regulatory	1	0.992	0.992	0.991
tflink	2	0.979	0.979	0.979
string neighborhood	3	0.952	0.951	0.950
string fusion	4	0.999	1.000	1.000
string cooccurence	5	1.000	1.000	1.000
string coexpression	6	0.729	0.731	0.729
string experimental	7	0.774	0.772	0.773
string database	8	0.996	0.996	0.995

Table 8: Neighbor recall at each gene's own degree, layer 1, one head per graph. Ordinary self-attention has no reason to place its top d_i mass on a gene's true neighbors, so these values are attributable to the penalty of Eq. 4.

Removing the penalty stops the model training. A matched pair of 30-epoch configurations exists for exactly this test, identical but for the graph term's weight. With the weight at 1.0, three replicates reach validation Pearson 0.464, 0.456 and 0.456. With it at 0.0, the three companion runs reach −0.007, −0.031 and a diverged run that returned no number, with training Pearson around 0.002.

That is a strong result and it needs a careful reading. It establishes that the graph term is load-bearing for optimization in this configuration. It does not establish that the graphs supply biological information the model uses, because a term carrying 99.99 percent of the loss is also doing whatever conditioning, scaling and implicit regularization that share implies. Separating the two would need a control with a comparable penalty against permuted or degree-matched random graphs, which has not been run.

6.4 Record-by-record agreement ✗

With per-record predictions available, the comparison can be made the way the DeWitt preprint makes it, between the predictions themselves rather than between their summary statistics.

The headline distinction from the preprint. DeWitt's networks correlated with the additive model at $r > 0.999$, so the network *was* the additive model. Here the correlation is 0.73 to 0.76, so the 010 transformer is genuinely computing something the additive model does not. That conclusion should be stated plainly, because it is the opposite of the preprint's finding and it is what the null was fit to test.

How reproducible the extra content is, measured directly. Run-to-run correlation between two transformer checkpoints is 0.79 to 0.82. An earlier version read that against the 0.73 to 0.76 agreement with the additive model and

	Pearson with		
Model	CGT M01	CGT M02	CGT M03
B1 additive ridge	0.752	0.726	0.759
B2 additive plus pair	0.759	0.734	0.765
B4 query pair only	0.729	0.706	0.735
B5 embedding MLP	0.788	0.784	0.810
CGT M01		0.788	0.821
CGT M02	0.788		0.804

Table 9: Correlation between models' test-set predictions, 37,673 records. The transformer does not reproduce the additive model, which is where 010 departs from the MULTI-evolve case. But two transformer training runs agree with each other at 0.788 to 0.821, barely more than a transformer agrees with the additive model at 0.726 to 0.759, and no more than it agrees with the nonlinear baseline at 0.784 to 0.810.

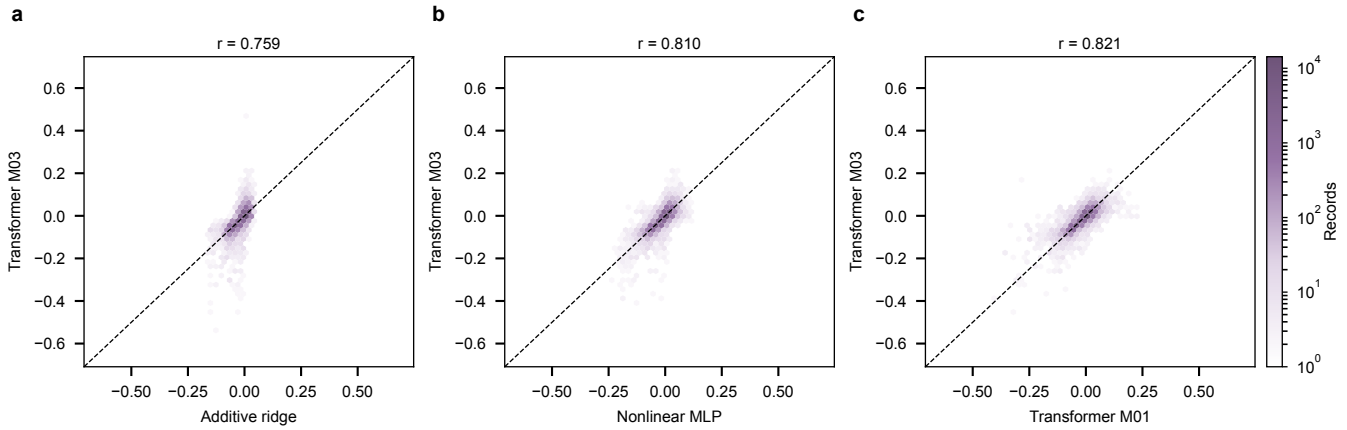

Figure 2: Test-set predictions of one model against another. The transformer is no closer to a second training run of itself than it is to a nonlinear baseline that has no graph and an eighth of the parameters.

concluded the non-additive content was largely irreproducible. That comparison is not sound, because both correlations are dominated by the additive part that every model reproduces, so neither number is about the non-additive content at all.

The direct measurement regresses each model's predictions on B1's and correlates what is left. On that scale the non-additive component is 43.5, 47.3 and 42.4 percent of each checkpoint's prediction variance, so it is a large share of what the model outputs. Between two checkpoints it correlates at 0.533, 0.584 and 0.564. Against the part of the label B1 also fails to explain it correlates at 0.236, 0.234 and 0.253, where B5's non-additive component manages 0.165.

So the non-additive content is neither noise nor stable. About half of it is shared between two training runs, and the shared part carries real signal about the residual label. The three runs are also not seed replicates in the strict sense: the training script fixes the seed at 42, M01 and M02 differ only in run-to-run nondeterminism, and M03 additionally changes the scheduler's first cycle length. A designed seed sweep has not been run.

The models are wrong in the same places. Correlating the residuals $y - \hat{y}$ of B1 and each checkpoint gives 0.907, 0.897 and 0.918. Almost all of the error is shared, so the transformer is not fixing a class of records the additive model gets wrong; it is making a modest, broadly distributed improvement.

The advantage is real and bounded. A paired bootstrap over test records, 2,000 resamples, puts the Pearson gain over B1 at +0.043 with a 95 percent interval $[+0.023, +0.063]$ for M01, +0.038 with $[+0.019, +0.058]$ for M02, and +0.055 with $[+0.034, +0.074]$ for M03. All three intervals exclude zero. B5 gains +0.025 with $[+0.017, +0.033]$. So the transformer beats a no-interaction null by a margin that is statistically established and is between two and six hundredths of a correlation.

Rankings diverge where selection happens. Among the 100 records each model calls most negative, B1 and M03 share 31, B5 and M03 share 45, and B1 and B5 share 46. At $K = 10{,}000$ all pairs exceed 0.91. The models agree about the bulk and disagree about the tail, which is the part a design-space selection reads.

Between two transformer checkpoints the overlap rises monotonically with K: 0.10 to 0.30 at $K = 10$, 0.39 to 0.47 at $K = 100$, 0.57 to 0.60 at $K = 1{,}000$, and 0.91 at $K = 10{,}000$. Reading the tail figure as a defect would be wrong. An extreme quantile is where the fitted surface is least constrained by data and where the label itself is noisiest, so two runs that agree about the bulk should be expected to disagree about the last hundred. The consequence for practice is unchanged, and it is a reason to ensemble rather than a reason to distrust the architecture.

Prediction spread. Test prediction standard deviations are 0.025 for B1, 0.029 for B5 and 0.036 to 0.038 for the checkpoints, against a test label standard deviation of 0.064. Every model is heavily shrunk toward the mean, as a squared-error fit on a noisy target should be.

6.5 The query-pair-disjoint split ✗

B4's 0.390 says most of Table 6 could be screen structure. The measurement that settles it is a split in which no query double appears in more than one part, and it has now been run.

The screen structure is exact, not approximate. Counting unordered gene pairs over all 376,732 records finds 739,315 distinct pairs, of which 420 occur five or more times. Those 420 cover 376,733 pair-instances. That is one more than the record count, so every record contains exactly one of the 420 recurring pairs and a single record contains two. Every record's most frequent pair occurs at least 200 times. The 420 pairs are the Kuzmin query doubles, and grouping records by them recovers the screen exactly rather than by a threshold that has to be defended.

One disjoint split, and then five. Assigning whole query pairs gives 285 train, 67 validation and 68 test pairs, or 80.03, 9.97 and 10.00 percent of records, with no query pair shared between any two parts. On that split the additive ridge reaches 0.174 test Pearson, against 0.135 on its own validation part. A 4-hundredth gap between two parts of one split is a warning that 68 query pairs is a small sample of screens, so the reported numbers below are five-fold cross-validation with the query pair as the grouping unit.

	Random split	Query-pair disjoint	
Model	test r	one split	5-fold mean ± sd
B0 train mean	0.000	0.000	0.000 ± 0.000
B4 query pair only	0.390	0.000	0.000 ± 0.000
B3 hierarchical mean	0.390	0.173	0.066 ± 0.039
B1 additive per-gene ridge	0.400	0.174	0.127 ± 0.033
B2 additive plus pair ridge	0.406	0.170	0.124 ± 0.034
B5 embedding MLP	0.426	0.150	0.058 ± 0.037
CGT	0.438 to 0.455	not measured, needs retraining	

Table 10: Held-out Pearson under the published random-over-records split and under a split that never shares a Kuzmin query double. The cross-validated column is the mean over five folds, each holding out a disjoint set of query pairs, with B5 additionally averaged over three seeds. B4 is structurally zero once query pairs are disjoint, because no test record carries a pair that has a coefficient.

Most of the additive null's score was screen identity. B1 falls from 0.400 to 0.127 ± 0.033. In explained variance that is r^2 from 0.160 to 0.016, so roughly nine tenths of what the additive null appeared to explain was carried by query doubles it shared with the training split. The fold-to-fold spread, 0.088 to 0.151, is wide enough that any single query-pair-disjoint split would misreport the number by several hundredths, which is why the single split in the middle column reads high.

The ladder inverts. Under the random split, interaction capacity bought a monotone climb from 0.390 to 0.455. Under the disjoint split the nonlinear baseline, which is the only baseline that can represent a three-way interaction, drops *below* the additive one, 0.058 ± 0.037 against 0.127 ± 0.033. It also early-stops at epoch 3 to 7 rather than epoch 20 to 30, having reached training Pearson 0.43 while held-out performance was already falling. Generalizing to an unseen query double is a different problem from generalizing to an unseen record of a seen query double, and the extra capacity that helped on the second hurts on the first.

What is not measured. The transformer has no row in Table 10. Its checkpoints were trained on the random split, so every record of a disjoint test set was potentially in their training data, and scoring them there would measure memorization. A transformer number for this split requires retraining, which Section 8 makes possible and which has not been run. Nothing here says the transformer's margin over B1 does or does not survive; it says the quantity all the random-split rows were competing on was mostly screen identity.

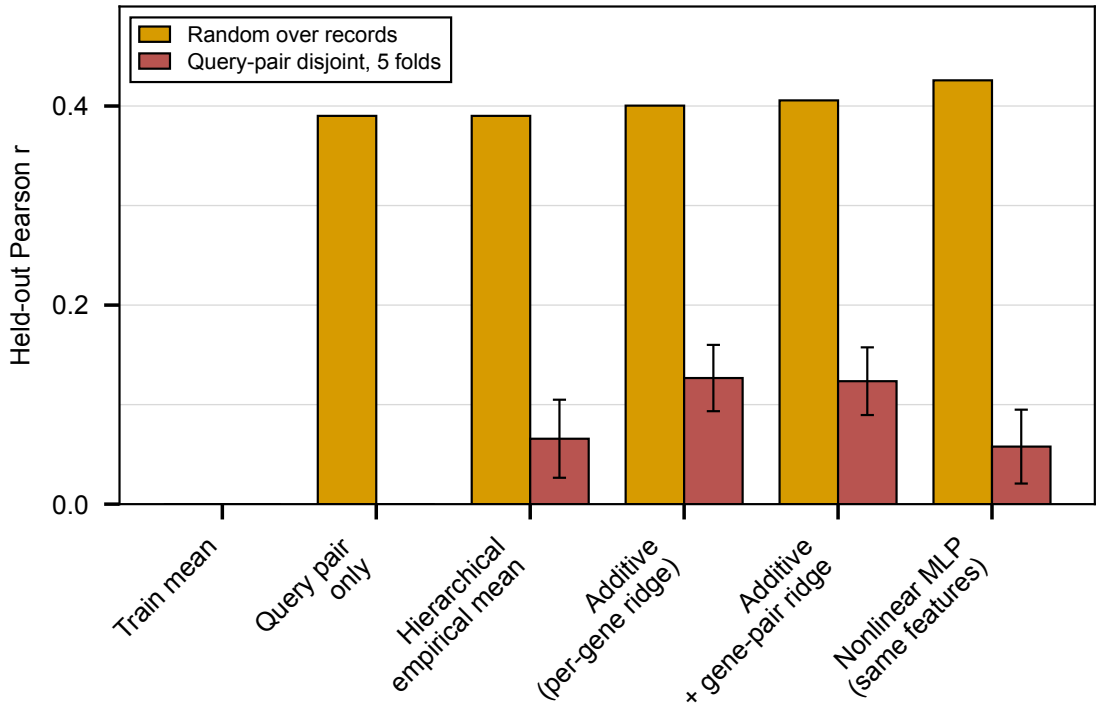

Figure 3: Each baseline under the published random-over-records split and under query-pair-disjoint folds. Error bars are the standard deviation across five folds. The query-pair-only model is structurally zero on the right, and the nonlinear model crosses below the additive one.

7 The design space, and a defect found there ✗

Figure 1 of the DeWitt preprint is a scatter of network predictions against additive predictions across the whole combinatorial space, and its force comes from the near-identity of the two rankings: the variants selected for construction were recoverable without the network. The same comparison on 010 gave a result that turned out not to be about additivity at all.

7.1 The comparison as run ✗

Checkpoint `c7671wgj` scored three separate 010 inference spaces:

- `inference_1`, 4,370,595 triples over a 526-gene panel,
- `inference_2`, 479,195 triples,
- `inference_3`, 465,735,532 triples genome-wide, which is the space the panel-12 and panel-24 selections consumed.

Scoring the same triples with B1 and streaming the 4 GB prediction file gives, over the 325,631,845 triples of `inference_3` whose three genes all carry an additive coefficient, a Pearson correlation of 0.0018 and a top-100 overlap of zero. Stratifying by the training support of each triple's least-observed gene does not account for it: even where all three genes have 800 or more training observations, $n = 867{,}669$, the correlation is 0.036.

On `inference_1` the same comparison behaves completely differently. B1 against the three checkpoints gives 0.548, 0.579 and 0.511.

7.2 One checkpoint scoring one triple two ways ✗

The model is a deterministic function of the perturbed gene set, so a triple scored twice by one checkpoint must receive the same number twice. It does not. The determinism is not in question and the resolution is that the two runs were not scoring the same triple, which Section 7.3 establishes. The symptoms are recorded first because they are what made the defect visible.

Deciding which run handles gene identity correctly needs a reference outside the transformer. The fitted coefficient vector $\hat{\beta}$ of Eq. 10 is one: it is estimated from measured training labels, keyed on systematic gene name, and independently

Comparison	n	Pearson
Same triple, `inference_2` vs `inference_3`	2,404 triples	1.0000
Same triple, `inference_1` vs `inference_3`	330 triples	0.295
Per-gene mean, `inference_2` vs `inference_3`	1,274 genes	0.800
Per-gene mean, `inference_1` vs `inference_3`	395 genes	0.037
Per-gene mean, `inference_1` vs `inference_2`	179 genes	0.167

Table 11: Agreement of checkpoint `c7671wgj` with itself across inference runs. Runs 2 and 3 are identical on shared triples, mean absolute difference 0.000000, so they share a pipeline. Run 1 is unrelated to them. Per-gene means average over hundreds of triples each, so sampling noise cannot explain 0.037.

validated at 0.400 test Pearson. If a run's predictions carry real gene identity, a gene's mean predicted effect should track $\hat{\beta}_g$.

Run	Own gene set	167 common genes
`inference_1`	0.579 (383 genes)	0.576
`inference_2`	0.048 (1,138 genes)	0.231
`inference_3`	−0.008 (3,180 genes)	0.174

Table 12: Pearson correlation between a run's per-gene mean prediction and the additive ridge coefficient for that gene. The right column restricts to the 167 genes all three runs score with at least 200 observations, so the different gene coverage of the three runs cannot explain the gap.

Restricted to identical genes, `inference_1` tracks gene identity more than three times as strongly as the two runs that produced the selection.

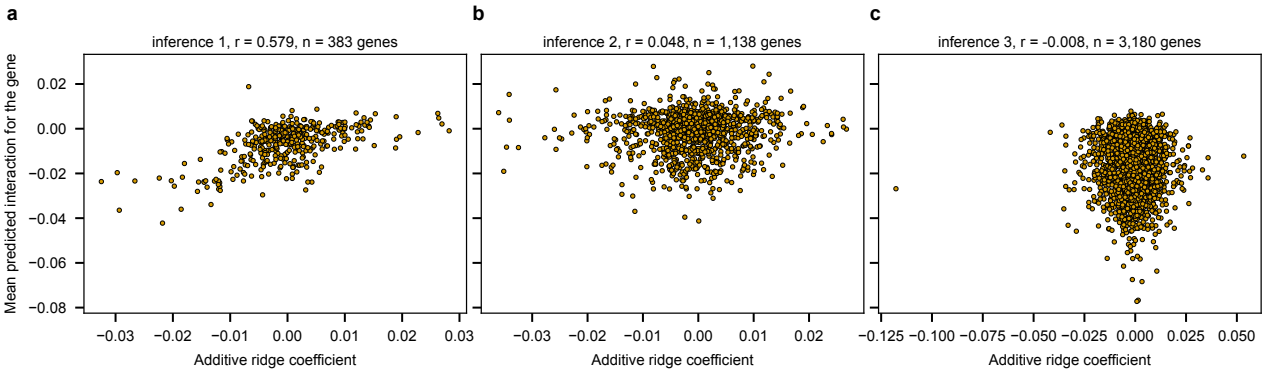

Figure 4: Per-gene mean prediction against the additive ridge coefficient, for each of the three inference runs of checkpoint `c7671wgj`. The additive coefficient is fit on measured training data and keyed on systematic gene name, so it anchors gene identity from outside the transformer.

7.3 The cause, confirmed ✗

The earlier version of this document labeled index misalignment a hypothesis. It has now been tested and it is the cause. The defect is a uniform shift of 28 in every gene index.

The test. The direct-scoring path of Section 6.2 is validated: it reproduces checkpoint `c7671wgj`'s recorded validation Pearson to 0.461917 against 0.461881. Re-scoring stored triples from each run through that path under two candidate index spaces identifies which space each run used.

The mechanism. The embedding table is indexed by the model's `gene_num`, fixed at 6,607 when the checkpoint was trained. The perturbed-gene indices in a batch are positions in the cell graph's sorted node list, and that graph is rebuilt at run time from whatever gene set the genome currently yields. Nothing asserts the two agree. Between January and February 2026 the genome gene set went from 6,607 to 6,579: the 28 missing entries are the mitochondrial open reading frames `Q0010` through `Q0297`, which sort first. Of the 6,579 genes present in both, not one keeps its index and every one shifts by exactly 28. `YAL001C` sits at row 28 in the space the checkpoint learned and was fed as row 0. Every `inference_2` and `inference_3` prediction is the model's answer for a different triple of genes, and 28 embedding rows were never addressed at all.

A second consequence of the same cause: a gene absent from the run-time gene set simply yields fewer matches, so a triple silently becomes a double or a single. That happened to 0.86 percent of `inference_2` rows and 2.76 percent of

Run	6,607-gene genome set	6,579-gene build set
`inference_1`	**0.999998**	0.13
`inference_2`	0.02	**0.999998**
`inference_3`	0.07	**0.999999**

Table 13: Pearson between each run's stored predictions and the validated scoring path, under each candidate index space. Every stored prediction is exactly reproducible, with mean absolute difference around 2×10^{-5}, which is the runs' half-precision autocast. Nothing here is stochastic, which answers the question of how a deterministic model disagreed with itself: it did not.

`inference_3` rows.

What this invalidates. Everything derived from the `inference_2` and `inference_3` prediction files, which is the panel-12 and panel-24 selections and the tables and figures built from them. The `inference_1` artifacts are unaffected, as is every number in Sections 6 and 6.4, which come from the labeled split through the validated scoring path.

The guard. The condition is now checked. The model raises if the cell graph's gene node count differs from the `gene_num` the checkpoint was built for, which turns a silent relabeling of the entire genome into a failure at the first forward pass. The deeper fix is to pin the index space with the build rather than reconstruct it from a live genome, which Section 8 takes up.

7.4 Seed instability in the selection tail, separately ✗

On `inference_1`, where predictions do track gene identity, the three checkpoints still disagree substantially about which triples are extreme. Checkpoint-to-checkpoint Pearson is 0.524 to 0.560, no higher than each checkpoint's 0.511 to 0.579 agreement with the additive model, and top-100 overlap between two checkpoints is 0.21 to 0.35. Two independent training runs of one architecture on one split nominate largely different top-100 triples, so a selection taken from a single checkpoint's tail is not reproducible across seeds even where the inference pipeline is sound.

8 Modernizing the 010 build ✗

Two of the obstacles in this document have the same root. Per-record predictions had to be rebuilt by hand rather than recomputed through the stock evaluation path, and the transformer has no row in Table 10, because the December 2025 build no longer loads under the current code.

8.1 What actually broke ✗

Two later changes invalidated it, and neither is about what was measured.

1. The perturbation-ontology refactor of 2026-07-08 renamed the deletion literals. Of the 1,130,196 stored perturbations, 1,008,761 carry `perturbation_type: "deletion"` with `deletion_type: "KanMX"`, which is now `sga_kanmx_deletion`, and 38,742 carry `deletion_type: "mean"`, which is now `mean_deletion`. The remaining 82,693 are allele and temperature-sensitive-allele perturbations and were not renamed. No record in this build is a `natMX` deletion.
2. The component-based media change of 2026-07-14 made `Media.is_synthetic` required. Every record and every reference in the build lacks it.

An earlier version of this document named only the first, and named the wrong replacement vocabulary for it. Both are pure vocabulary changes: neither alters which gene was perturbed, what was measured, or what the label is.

8.2 The migration, and why it copies rather than rebuilds ✗

Every published 010 artifact addresses records by position. The split file, the label table, the perturbed-gene index and the saved per-record predictions are all position-keyed, and the transformer checkpoints were selected against that split. A rebuild from the served graph returns records in a different order, so it would preserve the science and destroy the correspondence. Precedent is on record: carrying 010's splits into experiment 025 by a key that included the perturbation type matched 682 records of 376,732, because the type strings had been renamed in between, while a gene-set key matched all of them.

The migration is therefore a key-preserving copy. It rewrites the two vocabulary fields, writes a sibling build, and leaves the frozen original untouched, which it must, since the original's files are owned by the graph-build user and hardlinked into the graph tree. All 376,732 records migrate, all 376,732 validate against the current schema, and the check that the copy is faithful is that every position still carries the same three systematic gene names and the same label.

One thing the migration deliberately does not fix. The frozen build records YEPD at 30 degrees. The Kuzmin loaders now emit triple-mutant selection medium at 26 degrees, which is the corrected reading of the screen. Applying that here would change the experimental content of the very records whose published results the migration exists to preserve. A build with the corrected environment is a different data version, and it is a rebuild.

8.3 Which experiment ran on which data ✗

The migration creates a second 010 build, so the question of recording which version an experiment used is now concrete rather than hypothetical.

A commit hash does not answer it, for a reason this document supplies. The off-by-28 defect of Section 7.3 happened with no code change at all: the library was untouched between the runs, and what moved was the genome gene set the cell graph is rebuilt from. Two runs at the same commit scored different genes. Identity has to be carried by the built artifact, not by the code that built it.

What already exists is most of the machinery. Dataset builds get a `build_manifest.json` recording a contract fingerprint for every schema symbol the loader depends on, plus build time, host and commit, and staleness is judged by refingerprinting rather than by comparing commits. The gap is that experiment builds are not produced by that class and get no manifest, so neither 010 nor any later experiment build has one.

Three things would close it, in order of how much they are worth.

1. **Pin the index space with the build.** The gene ordering a checkpoint was trained against is the artifact whose drift caused the worst defect here, and it is currently recomputed at every use. Writing it into the build and loading it, rather than reconstructing it, makes the failure impossible instead of merely detected. The guard added in Section 7.3 is the detection half.
2. **Give experiment builds a manifest.** Extend the existing manifest to the experiment-build class, so a build records the schema contract it was made against and any run can name its data version.
3. **Record the source graph's identity.** A build made by querying the graph database stores the query but nothing about which database answered it. A build identifier from the server, recorded alongside the query, would make the chain complete.

The migrated build carries a `build_provenance.json` naming its source, that source's LMDB checksum, the exact field rewrites, and what was deliberately not changed. That is the first item's shape, written by hand for one build.

9 What this establishes, and what it does not ✗

9.1 Established ✗

The additive null had never been fit on 010, and now it has, on the same records, split and label the transformer used, with the ridge penalty chosen on validation and reported on test.

Every model in the comparison receives the same information, the identity of the three perturbed genes. That is a reading of the code rather than an assumption: the encoder is applied at batch size one to a strain-independent input, and the nine interaction graphs reach the prediction only through the weights that training left behind.

The transformer's margin over a model that cannot represent interaction is real and small on the published split.

The published split was measuring screen identity more than trigenic biology, and this is now measured rather than suspected. A model that never looks at the third gene reaches 97 percent of the additive null's score, and making query doubles disjoint drops the additive null from 0.400 to 0.127 ± 0.033 and inverts the ordering of the baselines.

The graphs are load-bearing during training. They carried 99.99 percent of the loss, their heads recover their graphs, and the matched pair of runs without the penalty does not train.

The design-space defect has a confirmed cause, a uniform 28-position gene index shift, and the artifacts it invalidates are enumerated.

9.2 Not established ✗

The reproduction is faithful but not bit-exact. The hand-built forward pass matches the recorded metrics to the fourth decimal, not exactly. The original runs used mixed bf16 precision and this one runs in fp32, which is the likely source, but that has not been verified by rerunning in bf16. Conclusions in Section 6.4 rest on correlations at the second decimal and are not sensitive to a fourth-decimal shift.

One seed of the nonlinear baseline enters the paired analysis. Table 9 uses B5 seed 0. The three seeds agree closely on aggregate metrics, but their record-level agreement with each other was not measured, so the B5 column should be read as one draw.

The transformer has not been measured on the disjoint split. Table 10 has no transformer row. Its checkpoints trained on the random split, so scoring them on held-out query pairs would measure memorization. Whether the transformer's +0.04 margin survives a disjoint split is not measured, in either direction.

Whether the graphs carry information is separate from whether they are load-bearing. The runs without the graph penalty fail to train, and the penalty was almost the entire loss. Both facts are recorded. Neither separates the graphs supplying biological structure from a large auxiliary term supplying conditioning that the optimization needs. The control that would separate them, a comparable penalty against degree-matched random graphs, has not been run.

The graph ablation is short and unreplicated in one arm. The matched pair ran 30 epochs against the 49 to 64 of the reported checkpoints, and the arm without the penalty produced two numbers and one diverged run rather than three.

Nothing here bears on the architecture's merits elsewhere. The reduction in Section 3.4 is specific to the 010 configuration, where the masking and propagation arms are off and the perturbation transform is one block. A configuration that switches those on would have a strain-dependent encoder path and would need its own reading.

9.3 Next ✗

1. Train the transformer on the query-pair-disjoint split and fill the empty row of Table 10. The migrated build makes this runnable, and it is the measurement the rest of this document is now waiting on.
2. Regenerate the panel-12 and panel-24 selections from a correctly indexed inference run. The current selections are invalid, not merely uncertain.
3. Run the graph ablation against degree-matched random graphs, to separate the graphs carrying structure from a large penalty carrying conditioning.
4. Pin the gene index space with the build rather than reconstructing it from a live genome, which is the fix that would have prevented the defect the guard now only detects.
5. Report an ensembled ranking for any future selection rather than one checkpoint's tail, given that two training runs share 0.39 to 0.47 of their top 100.
6. Give experiment builds the manifest that dataset builds already get, so a run can name its data version.